

Chapter 5: Procedures

runtime stacks are an essential part of the mechanism for calling and returning from Procedures

Runtime Stacks:

- is a memory array managed by CPU using ESP[extended stack pointer] register
- esp register holds a 32-bit offset into some location on the stack
- The esp is indirectly modified by the instructions: call, ret, push and pop
- esp always points to the last value to be added to, or pushed on, the top of the stack
- Stacks in COAL are opposite to stacks in DSA as the stacks in COAL are from top to bottom i.e the push operation decrements the esp register and the pop function increments the esp register

Stack Applications:

- The stack provides temporary storage for local variables inside subroutines
- when calling a subroutine, you pass input values called arguments by pushing them on a stack

Push Instruction:

- The push instruction first decrements esp then copies the source operand into the stack
- Allowed Formats:
 - * push register/mem16
 - * push register/mem32
 - * push imm32

Pop Instruction:

- The pop instruction first copies the contents of the stack into a 16/32 bit register then increments the esp
- Allowed Formats:
 - * pop register/mem16
 - * pop register/mem32

Pushfd and Popfd Instructions:

the pushfd instruction pushes the 32bit eflags registers on the stack, and popfd pops the stack into eflags

Pushad, popad and pusha, popa Instructions:

- the pushad pushes all the 32 bit general purpose registers into the stack in the following order:
 - * eax, ecx, edx, ebx, esp, ebp, esi, edi.
- the popad restores the values of the 32 bit register from the stack
- the pusha pushed all the 16 bit general purpose registers into the stack in the following order:

* ax, cx, dx, bx, sp, bp, si, di

- the popa restores the values of the 16 bit registers from the stack.

x -----
----- x

A Procedure is equivalent of a C++ function.

Defining of a procedure:

- (name of the procedure) Proc
 code
 code
 code
 ret <-- ret forces the cpu to return to the location
from where the procedure was called
 (name of the procedure) endp

* The labels in the procedure will only work if the jump statement and the destination label are in the same procedure.

* This limitation can be bypassed by declaring the destination label as global by means of a double colon(::)

* The call Instruction is used to call the procedure to begin its execution at a new memory location
 the call instruction pushes its return address on the stack and copies the called procedure's address into the instruction pointer

* when the procedure is ready to return, its ret function pops the return address from the stack into the instruction pointer.

Uses operator:

- the uses operator, coupled with the proc directive, lets you list the names of all registers modified within the procedure

- uses tells the assembler to do two things
 * First, to generate push instructions that save the registers on the stack at the beginning of the procedure

 * Second, to generate pop instructions that restores the register's value at the end of the procedure